ECOLE POLYTECHNIQUE DE BRUXELLES

2026

GPU Computing

Mini project
Study of kernel's performance

Niccolò Benedetto

Paolo Bordis

Prof. Daniele Bonatto

PhD Eline Soeten

March 2026

Contents

1 Introduction **1**

2 Experimental Setup **2**

2.1 Computational Resources 2

2.2 Experiment 2

3 Comparison **5**

3.1 Analysis 6

1 Introduction

This document investigates the performances of CUDA kernels by considering the vector addition as main operation, comparing `float` and `double` precision. This study has as main goal the evaluation of metrics like computation throughput, memory bandwidth and compute intensity across different configurations. In fact experiments were conducted by varing vector sizes, the number of element mapped per thread (mapping) and additional computational workloads. We analyze then the impact of precision, arithmetic intensity on GPU performance by means the roofline model as a reference for the performance limits.

2 Experimental Setup

2.1 Computational Resources

Table 2.1: GPU specifications

Device name	NVIDIA GeForce GTX 1080 Ti
Compute capability	6.1
Max Threads per block	1024
Max Threads per Multiprocessor	2048
Shared Memory per block	48 KB
Shared Memory per Multiprocessor	96 KB
Registers per Block/Multiprocessor	65536
Max Grid Size	[2147483647 65535 65535]
Max Block Dimensions	[1024 1024 64]
Single precision FLOP/s (FP32)	3914.99 GFLOP/s
Double precision FLOP/s (FP64)	1957.49 GFLOP/s
#Multiprocessor (SM)	28
Processor Clock Rate	1.607 GHz
Threads per Warp	32
Global Memory Bandwidth	484.44 GB/s
Global Memory Size	10.9999 GB
Constant Memory Size	64 KB
L2 Cache Size	2816 KB
Memory Clock Rate	5505 MHz

2.2 Experiment

The experiment presented in this report aims to compare two different datatypes for the same operation. The code takes into account here `float` and `double` datatypes in order to compare them through the vector addition operation.

The setup considers two initial arrays of size variable (two per type) filled with the same rule as follow. For the `double` type:

$$a[i] = i \times 0.0001$$

$$b[i] = i \times 0.0002$$

and for the `float` type:

$$a2[i] = i \times 0.0001f$$

$$b2[i] = i \times 0.0002f$$

Then the kernels perform the sum for each index and store it in the new array `c`.

Neverthless, before doing that, the kernels consider as well the cos of the sin of the multiplication between the counter of the loop (that takes into account the parameter `k` that rule the additional workload) and the term `0.01`. Speaking about code, the kernel performs something like:

Listing 2.1: CUDA kernel for vector addition with additional computation

```
__global__ void addKernel_double(double* c, const double* a, const
    double* b, int N, int j, int k)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int totalThreads = blockDim.x * gridDim.x;

    for (int elem = 0; elem < j; elem++) {
        int i = idx + elem * totalThreads;
        if (i < N) {
            double val_a = a[i];
            double val_b = b[i];
            double res = val_a + val_b; //1 op

            for (int loop = 0; loop < k; loop++) {
                res *= cosf(sinf(loop * 0.01));
```

```
15                }
16
17                c[i] = res;
18            }
19        }
20    }
```

In addition we see that the mapping technique was used to process several elements by the same thread, using a stride. The stride is of the size of a block (compute dinamically) and the number of elements processed depends on the mapping parameter j. This overview so far reported represents our setup of the kernels computation.

The comparison has been made over different vector sizes (65536, 262144, 1048576, 4194304, 16777216). We use multiples of 1024, since for a GPU 1080 the maximum number of threads per block is indeed 1024. In addition, cosidering this just said, we set a thread count of 64 thread per block. The mapping range is (1, 4, 16, 32, 64, 256, 512) and the amount of additional calculations (cos and sin operations) is ruled by the parameter k in the range (1, 10, 100, 1000, 10000, 100000).

3 Comparison

The compute and memory performance are defined through different metrics introducet across the labs. In particular we consider:

$$\text{Memory Throughput/Bandwidth} = \frac{\text{\#bytes read/written per kernel}}{\text{runtime for the kernel}} [GB/s]$$

$$\text{Computation Throughtput/Operation Per Second} = \frac{\text{\#operation done per kernel}}{\text{runtime for the kernel}} [GOPS]$$

$$\text{Computation Intensity} = \frac{ComputationThroughtput}{\text{Memory Throughput}} [OPS/byte]$$

Every experiment was averaged over 10 repetitions.

3.1 Analysis

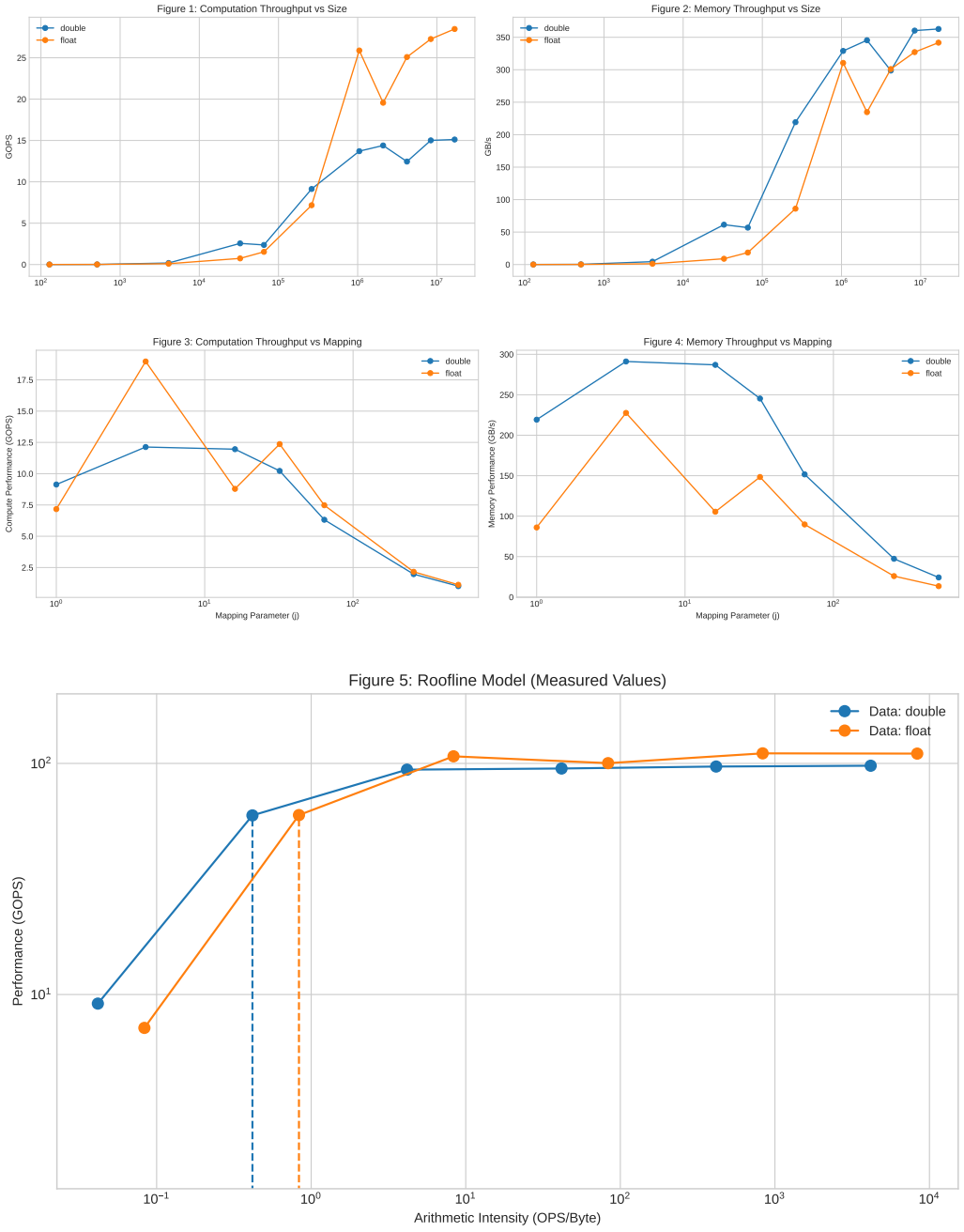


Figure 3.1: Performance results for kernel with k=1 and j=256.

In the plot Figure 3.1 - first plot the `double` compute performance is basically always below than the `float` one. The reason can be attributed to the throughput of the GPU (see Table 2.1) where it specified that the performance is lower for `double` precision operations. Neverthless we observe that in the second graphic the `double` memory performance is higher. Even if it looks like illogical we need to look carefully at the definition of memory throughput. The `double` type is 8 bytes long against 4 byte for the `float`. Since they have approximately the same execution time (`double` is slightly slower), at the end of the day the `double` moves more data per seconds.

In the mapping plot of Figure 3.1 we see that `float` performs better than `double`. The catch is the same for the precedence reasoning: the maximum compute throughput is higher for FP32 than for FP64. Hence for a small amount of elements mapped to the same thread (1-2), compute performance is high, meaning that memory latency is low, and compute instructions can be execute efficiently. At higher mapping (4-16) compute performance drops significantly, likely due to increased memory latency from strided access. As result GPU spends less time doing computations and more time waiting for data. At very high mapping (64-256) compute performance partially recovers, suggesting that the kenrnel schedules more independent memory requests efficiently.

In the fourth plot we observe that for a small amount of elements mapped to the same thread (1-6) memory perfomance remains high, as each thread is still accessing data within close memory regions, keeping the access coalescing. When the number of the mapped increase, memory throughput drops significantly, showing that strided access causes no more coalesced memory transactions, leading to wasted memory bandwidth. In addition we see that `double` precision still achieves higher memory throughput than `float` because each transaction loads twice as much data per read, even though access are strided.

The roofline model is presented in the fifth plot of Figure 3.1. Basically it takes the performance (GOPS) in function of the compute intensity (OPS/byte). The left part (separated by the dotted line) is the memory bound region, instead the part on the right is the compute bound region. The `float` have higher compute performance in function of compute intesity (peak at about 110 GOPS). Instead the `double` performs slightly less than the `float` with a peak at about 100 GOPS. All this is once again due to the FP32 operations being faster than the FP64.

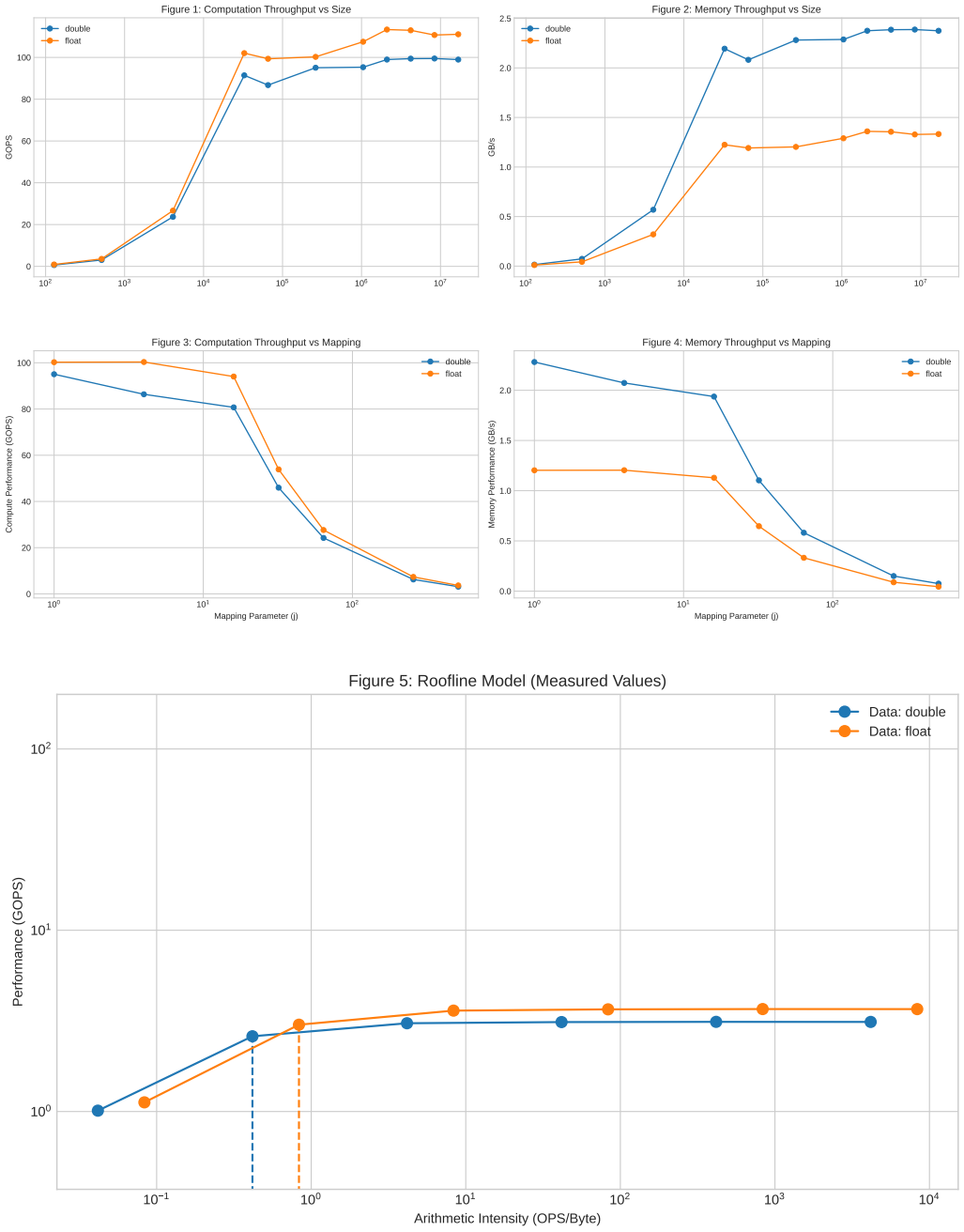


Figure 3.2: Performance results for kernel with k=1000 and j=512.

We report now the insights when we increase the workload of the kernel (parameter k). When we consider a workload of 1000 additional iteration in the kernel is interesting look at how the values of the metrics choosen change. Focusing on the memory throughput graphics (second and fourth graphic of Figure 3.2) we observe that the values of the memory performance drop drastically (from 300-350 GB/s earlier to slightly 2.5 GB/s now). Contrary the compute performance values (giga operations per second) become larger (to 17.5/25 GOPS to 100/130 GOPS). That's because we have already saturated the memory by moving the kernel to a compute setup.